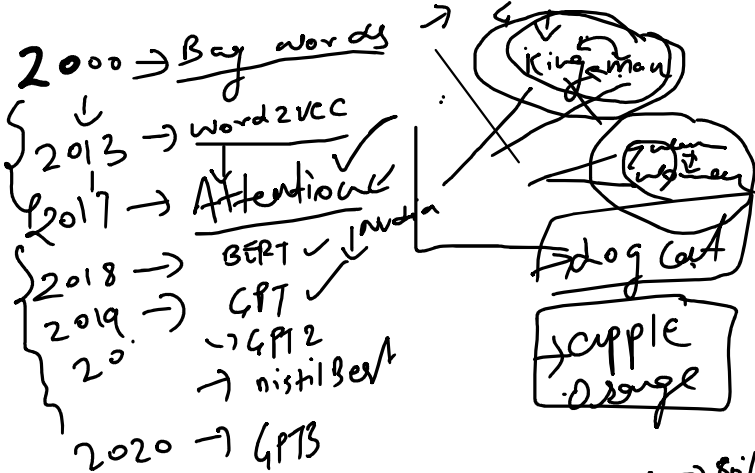


There is a cat.



2022 → FlanT5 → Stackoverflow

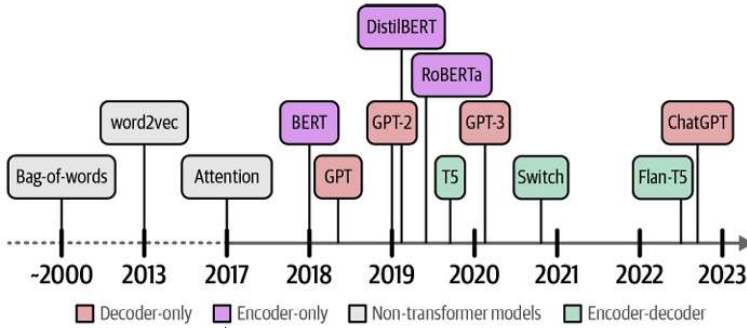
2023 → ChatGPT

→ LLM Application

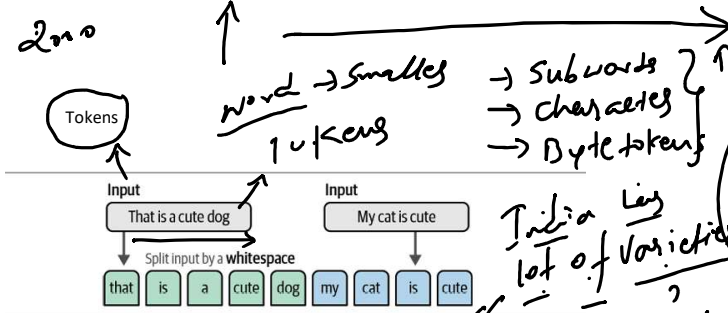
↓ sentiment classif.

RAG

↓ static AI



2nd



LLM

- Text
- Visual
- images

(1000 words)

embeddings

	cats	puppy	houses	apple	baby
animal	.91	.93	-.56	-.67	.01
newborn	-.11	.71	-.32	-.1	.90
human	.19	.36	.31	.29	.87
...	...	...	...	...	...
plural	.94	-.82	.94	-.51	-.11
fruit	-.51	-.91	-.5	.89	-.51

Number of dimensions (properties)

Numbers → computers

7 words Token

+1 +1 = 9 Token

Embedding → 25 Token

Compute a high accuracy May.

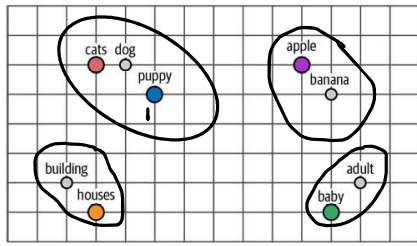
medical

→ buy computer

... ..

plural	94	-82	94	-51	-11
fruit	-51	-91	-5	89	-51

(properties)



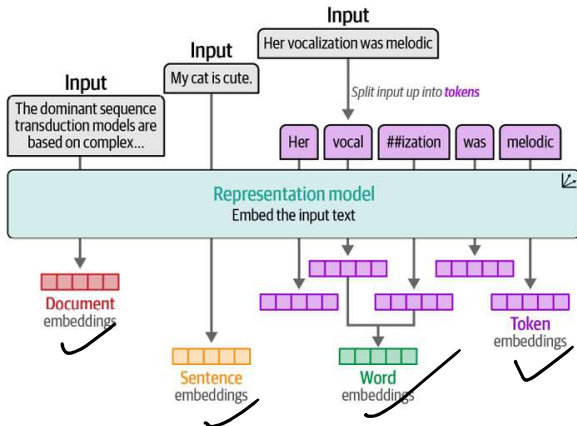
Multidimension representation

Embedding →

NLP

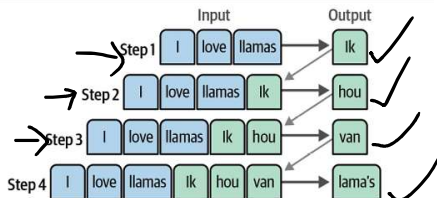
Embeddings ↑ word2vec → GloVe → Google

Token → embedding



Type of embeddings

Basic for AI



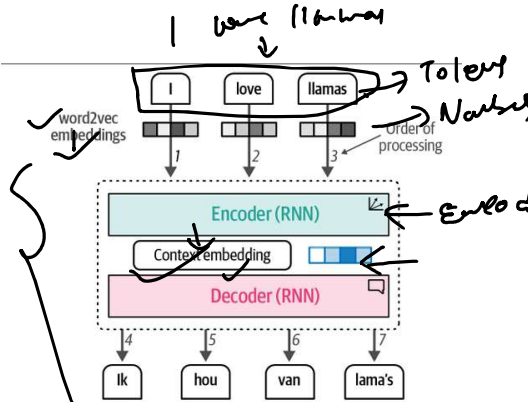
How it works

next token output

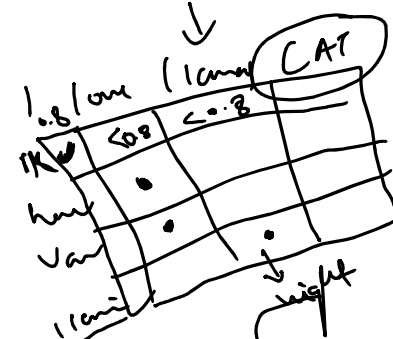
chatGPT

Self Attention

Attention example



I am going to play and will be late



who will be late

Attention

2017 → Attention

Persona

Describe what role the LLM should take on. For example, use "You are an expert in astrophysics" if you want to ask a question about astrophysics.

Instruction

The task itself. Make sure this is as specific as possible. We do not want to leave much room for interpretation.

Context

Additional information describing the context of the problem or task. It answers questions like "What is the reason for the instruction?"

Format

The format the LLM should use to output the generated text. Without it, the LLM will come up with a format itself, which is troublesome in automated systems.

Audience

The target of the generated text. This also describes the level of the generated

zero shot

→ I do not like coffee

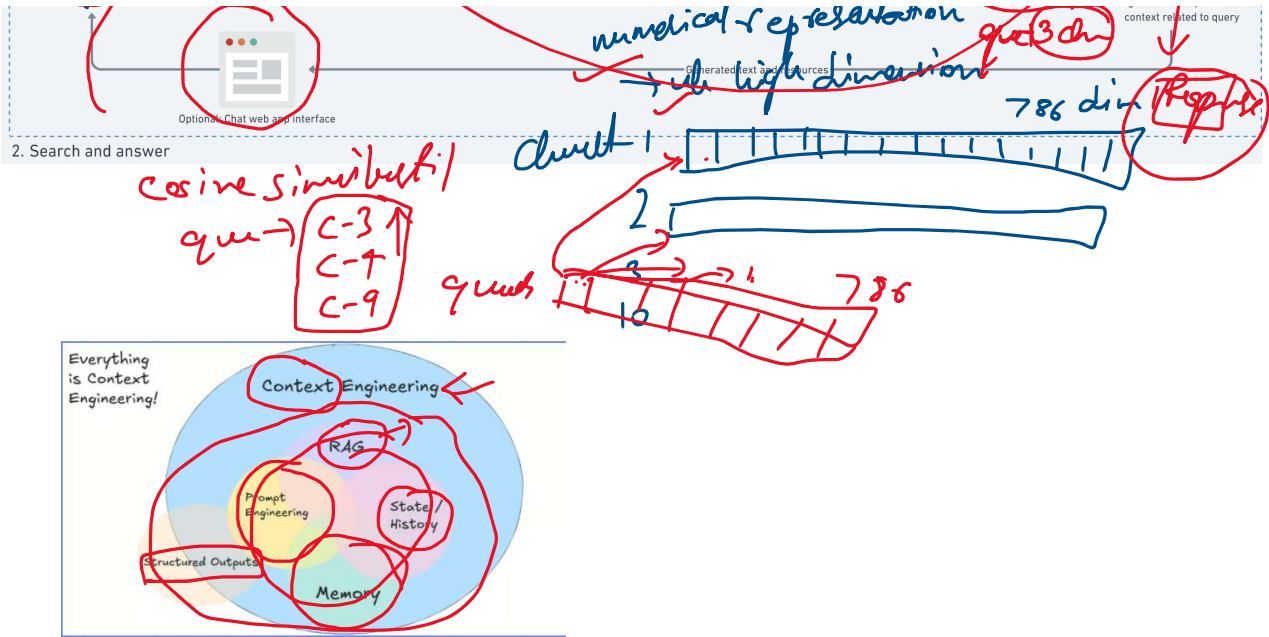
few shot

I do not like hot beverages

Now I do not like hot tea

→ I do not like coffee
→ -ve
+ve

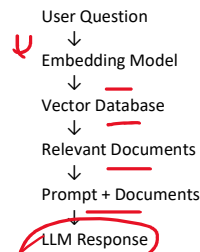
~~Sentences~~ → -ve
Now I do not like that
+ve
↓
Sentences
→ -ve



Example Embedding Models

- sentence-transformers
- BERT
- Instructor-XL

Traditional databases store structured data.
Vector databases
store high-dimensional vectors.
allows similarity search:



Similarity Search Methods

Explain these algorithms:

Method	Description
Cosine similarity	Measures angle between vectors
Euclidean distance	Measures distance
Dot product	Measures alignment

These power semantic search engines.

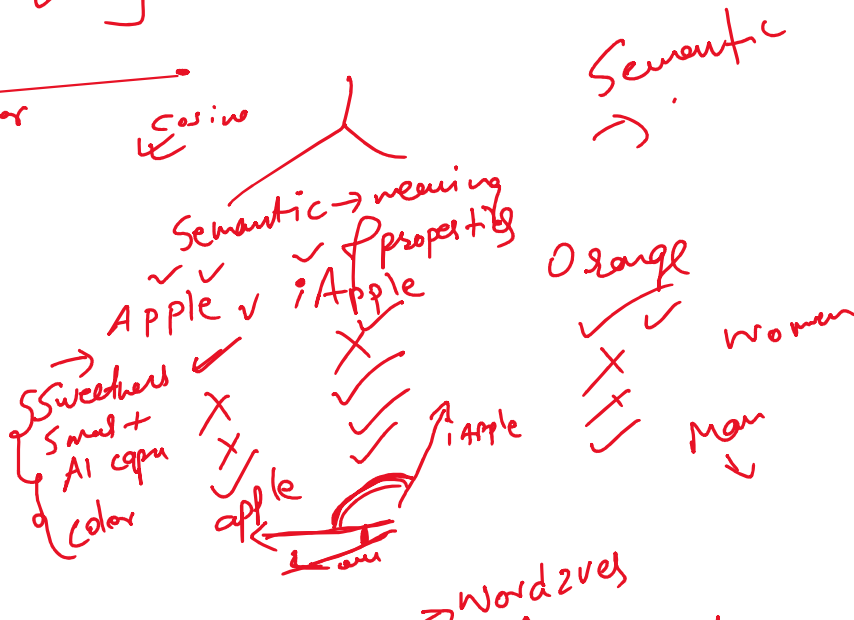
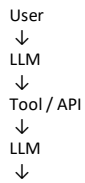


- FAISS
 - Chroma
 - Pinecone
 - Weaviate
- vector

Streaming LLM Responses

- faster user experience
- feels interactive

Function Calling / Tool Use



Tool / API
↓
LLM
↓
Answer

Guardrails and Safety

Guardrail Techniques

Technique	Purpose
Input filtering	Block unsafe prompts
Output filtering	Detect harmful responses
Moderation models	Detect toxic content
Prompt constraints	Limit model behavior

Model Evaluation

Metric	Purpose
BLEU	text similarity
ROUGE	summarization quality
Perplexity	language fluency

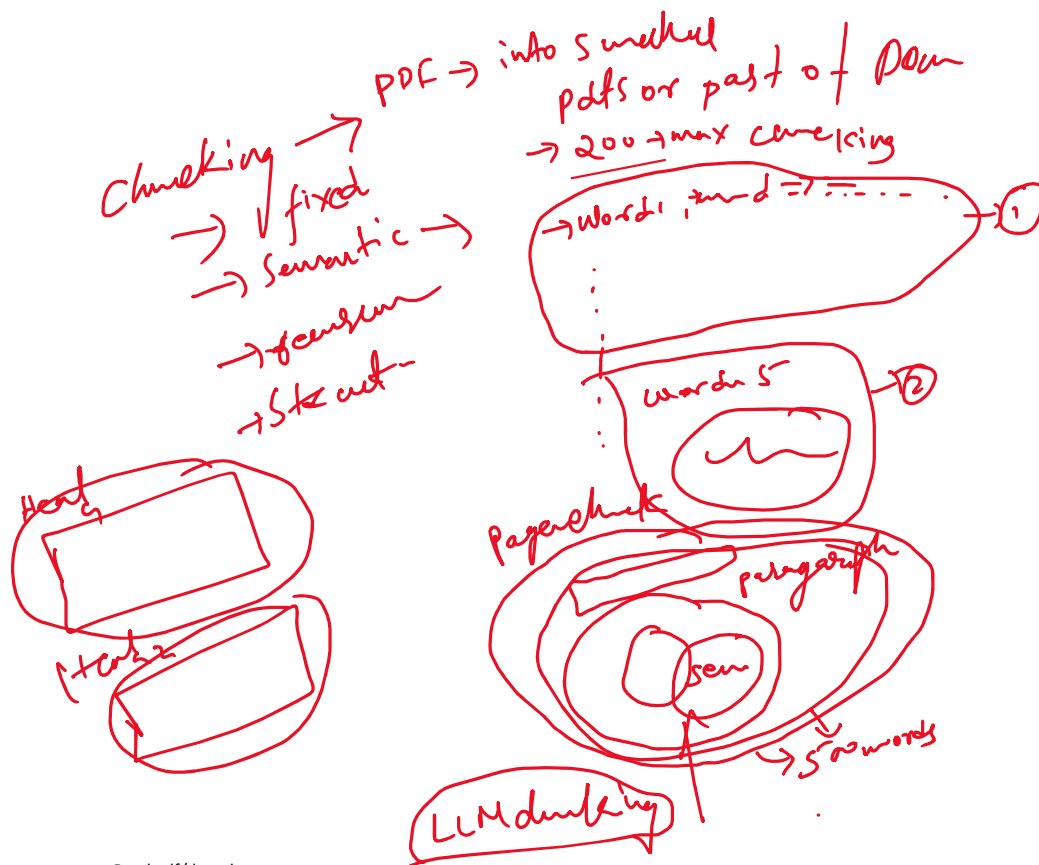
Basic Chatbot

↓
Chat + Memory
↓
Chat + Database
↓
Chat + RAG
↓
AI Assistant
↓
AI Agents

Retrieval

Augmented

Generation



Read pdf/data document
Extract data
Chunk it - fixed, semantic, recursive, structural, LLM Chunking

Legal Domain Example:	You're building a system to answer questions about a country's laws. Laws are typically divided into articles and sections.
Finance Domain Example:	You have earnings call transcripts and want to do Q&A on them. An earnings call transcript has an initial presentation, then a Q&A section. It might not have clear headings for each question.
Healthcare Domain Example:	You are processing patient electronic health records (EHRs) for a clinical assistant. EHRs have semi-structured fields (e.g., "Chief Complaint", "History of Present Illness", "Lab Results", "Assessment and Plan").
Education Domain Example:	Suppose you have a set of lecture transcripts or long video subtitles that you want to use in a QA system for students.

Which embedding models to use

1. **Size of input** - If you need to embed longer sequences, choose a model with a larger input capacity.
2. **Size of embedding vector** - Larger is generally a better representation but requires more compute/storage.
3. **Size of model** - Larger models generally result in better embeddings but require more compute power/time to run.
4. **Open or closed** - Open models allow you to run them on your own hardware whereas closed models can be easier to setup but require an API call to get embeddings.

1. **Pinecone**: private, cloud based, very popular
2. **pgvector**: Postgres vector search extension, useful if you have already data in Postgres.
3. **Faiss**: A library for efficient similarity search and clustering of dense vectors.
4. **Milvus**: An open-source similarity search engine for embedding vectors.
5. **Weaviate**: Open source vector database written in Go.
6. **Qdrant**: Open source vector search engine written in Rust.
7. **Chroma**: Yet another vector database